

■ BitQuest

Explorador de Matrices con C y NASM

Plan de trabajo · 3 integrantes · Windows x64 · 4 días

Este documento organiza el desarrollo de BitQuest en 4 días, con tareas paralelas de dificultad equivalente para tres integrantes. Cada integrante tiene archivos propios y puede avanzar desde el día 1 sin depender de los demás, integrándose con el resto mediante headers compartidos en las primeras horas.

Integrante	Rol	Archivos clave	Puntos en criterios
Integrante 1	Motor del juego y renderizado	juego.c · juego.h · assets.h · colores.h	~40 pts (ventana, movimiento, llave/puerta, monedas)
Integrante 2	Funciones NASM + build system	rutinas.asm · rutinas.h · build.bat · README.md	~30 pts (5 funciones NASM) + build/README
Integrante 3	Mapas, carga y pantallas especiales	mapas/*.txt · mapas.c · mapas.h · main.c	~18 pts (niveles, matrices) + puntos extra

Integrante 1 — Motor del juego y renderizado

Es el núcleo del juego. Implementa todo lo que el jugador ve y hace: renderizado de la ventana visible, movimiento, lógica de objetos y pantallas de resumen. Trabaja principalmente en `juego.c` y `juego.h`.

Archivos bajo su responsabilidad

Archivo	Contenido
<code>juego.h</code>	Structs Jugador y Nivel, constantes FILAS/COLS/VIS/NUM_NIVELES, prototipos
<code>juego.c</code>	Implementación de todas las funciones del juego
<code>assets.h</code>	CHAR_* y VISUAL_* para la representación del mapa (ya definido)
<code>colores.h</code>	Códigos ANSI para colores en consola (ya definido)

Tareas en orden de implementación

Tarea 1: `imprimir_ventana()`

- Recibe puntero a Nivel y a Jugador.
- Calcula `cam_fila = jugador.fila - VIS/2`, `cam_col = jugador.col - VIS/2`.
- Clampea la cámara para no salir del mapa.
- Para cada celda visible, imprime el VISUAL_* correspondiente con su COLOR_* de colores.h.
- Limpia la pantalla antes de imprimir (`system("cls")` en Windows).

Prototipo:

```
void imprimir_ventana(Nivel* n, Jugador* j);
```

Tarea 2: `mostrar_hud()`

- Imprime una línea de estado encima o debajo del mapa.
- Muestra: número de nivel, monedas recolectadas / total, pasos, si tiene llave (Sí/No).
- Usa colores ANSI para resaltar cada dato.

Prototipo:

```
void mostrar_hud(Jugador* j, int nivel_num);
```

Tarea 3: `mover_jugador()`

- Recibe la tecla presionada (w/a/s/d) y calcula nueva_fila y nueva_col.
- Llama a `validar_movimiento()` de NASM (puede usar stub que devuelva 1 mientras Integrante 2 termina).
- Si es válido: llama a `detectar_objeto()` para verificar qué hay en esa celda.
- Maneja: recoger moneda (M→'.', monedas++), recoger llave (K→'.', tiene_llave=1),
- cruzar puerta solo si tiene_llave, detectar salida (E) para terminar nivel.
- Incrementa pasos en cada movimiento exitoso.

Prototipo:

```
void mover_jugador(Nivel* n, Jugador* j, char tecla);
```

Tarea 4: mostrar_resumen_nivel()

- Imprime el bloque de resumen al terminar cada nivel.
- Formato: monedas recolectadas / total, pasos del nivel.

Prototipo:

```
void mostrar_resumen_nivel(Jugador* j, Nivel* n);
```

Tarea 5: mostrar_resumen_final()

- Imprime el resumen global al completar los 4 niveles.
- Muestra: monedas totales, pasos totales, niveles completados, puntaje final.
- El puntaje viene de calcular_puntaje() de NASM.

Prototipo:

```
void mostrar_resumen_final(int mon, int mon_max, int pasos, long puntaje);
```

Punto de integración

- ⇒ Recibe rutinas.h del Integrante 2 el día 1. Puede usar stubs que devuelvan 1 hasta que NASM esté listo.
- ⇒ Recibe mapas.h del Integrante 3 el día 1. Puede probar con un mapa hardcodeado temporal.
- ⇒ No toca main.c — ese archivo pertenece al Integrante 3.

Integrante 2 — Funciones NASM + Build System

Es el integrante más crítico para la calificación: las 5 funciones NASM valen 30 puntos directos. También construye el sistema de compilación que usará todo el equipo.

Archivos bajo su responsabilidad

Archivo	Contenido
<code>rutinas.h</code>	Prototipos C de las 5 funciones NASM — se entrega el día 1
<code>rutinas.asm</code>	Implementación de las 5 funciones en NASM 64 bits (win64)
<code>build.bat</code>	Script de compilación NASM + GCC para Windows
<code>README.md</code>	Instrucciones de instalación, compilación y ejecución

■ *Convención de llamadas en Windows x64 (diferente a Linux): los parámetros llegan en RCX, RDX, R8, R9 (no RDI/RSI como en Linux). El retorno va en RAX. Preservar: RBX, RBP, RDI, RSI, RSP, R12–R15.*

Tareas en orden de implementación

#	Función	Parámetros (Windows x64)	Retorna	Pts
1	<code>contar_caracter</code>	<code>RCX=mapa, RDX=total_celdas, R8=char</code>	RAX = veces que aparece el char	10
2	<code>validar_movimiento</code>	<code>RCX=mapa, RDX=cols, R8=nueva_fila, R9=nueva_col</code>	RAX = 1 válido / 0 bloqueado	8
3	<code>calcular_puntaje</code>	<code>RCX=monedas, RDX=pasos, R8=niveles</code>	RAX = puntaje calculado	5
4	<code>detectar_objeto</code>	<code>RCX=mapa, RDX=cols, R8=fila, R9=col, [stack]=char</code>	RAX = 1 encontrado / 0 no	4
5	<code>contar_libres</code>	<code>RCX=mapa, RDX=total_celdas</code>	RAX = celdas con '.'	3

Estructura de `rutinas.asm`

```
section .text
global contar_caracter, validar_movimiento, calcular_puntaje
global detectar_objeto, contar_libres
; Función 1: contar_caracter
; RCX=mapa RDX=total R8=char_buscado
contar_caracter:
xor rax, rax ; contador = 0
xor r9, r9 ; índice = 0
.loop:
cmp r9, rdx
jge .fin
movzx r10, byte [rcx + r9]
cmp r10, r8
jne .sig
```

```
inc rax
.sig:
inc r9
jmp .loop
.fin:
ret
```

build.bat

```
@echo off
nasm -f win64 rutinas.asm -o rutinas.obj
gcc main.c juego.c mapas.c rutinas.obj -o BitQuest.exe
if %errorlevel%==0 (
echo [OK] Compilacion exitosa.
BitQuest.exe
) else (
echo [ERROR] Revisa los mensajes anteriores.
)
```

Punto de integración

- ⇒ Entrega rutinas.h con los 5 prototipos el día 1 — desbloquea al Integrante 1.
- ⇒ El build.bat compila main.c + juego.c + mapas.c + rutinas.obj en un solo comando.
- ⇒ Si el Integrante 1 o 3 encuentran un bug al compilar, este integrante ajusta el build.bat.

Integrante 3 — Mapas, Carga y Pantallas Especiales

Construye el contenido del juego (los 4 niveles), el sistema de carga desde archivos .txt y las pantallas de inicio y victoria. Conecta todo en main.c.

Archivos bajo su responsabilidad

Archivo	Contenido
mapas/nivel1.txt	Mapa nivel 1 · 60×60 · laberinto sencillo
mapas/nivel2.txt	Mapa nivel 2 · 60×60 · intermedio
mapas/nivel3.txt	Mapa nivel 3 · 60×60 · avanzado
mapas/nivel4.txt	Mapa nivel 4 · 60×60 · punto extra (>3 niveles)
mapas.h	Prototipo de cargar_mapa() — se entrega el día 1
mapas.c	Implementación de cargar_mapa()
main.c	Bucle principal, pantalla_inicio(), pantalla_victoria()

Tareas en orden de implementación

Tarea 1: Diseño de los 4 mapas .txt

- Cada mapa debe tener exactamente 60 filas y 60 columnas de caracteres.
- Usar solo: # . P M K D E (sin espacios extra al final de línea).
- Cada mapa tiene exactamente: 1 P, al menos 1 K, 1 D, 1 E, varios M.
- Tip: diseña el laberinto en cuadrícula, luego transcríbelo. Verifica el ancho con el contador de columnas de VS Code.
- Nivel 1: pasillos amplios, pocas bifurcaciones. Nivel 4: el más retador.

Nota técnica:

Herramienta: VS Code con fuente monoespaciada, contador de columnas en la barra inferior

Tarea 2: cargar_mapa()

- Abre el .txt con fopen(), lee línea por línea con fgets().
- Copia cada carácter al arreglo mapa[fila][col] ignorando el '\n' final.
- Devuelve 1 si cargó correctamente, 0 si no encontró el archivo.
- Si falla, imprime un mensaje de error claro para facilitar el debug.

Nota técnica:

```
int cargar_mapa(const char* ruta, char mapa[][COLS]);
```

Tarea 3: pantalla_inicio()

- Limpia pantalla, imprime el título de BitQuest con arte ASCII y colores.
- Muestra las instrucciones básicas de movimiento (W/A/S/D, Q para salir).
- Espera que el usuario presione Enter para comenzar.
- Punto extra directo: pantalla de inicio.

Nota técnica:

```
void pantalla_inicio();
```

Tarea 4: pantalla_victoria()

- Se muestra al completar los 4 niveles.
- Arte ASCII de victoria con colores, mensaje de felicitación.
- Muestra el puntaje final recibido como parámetro.
- Punto extra directo: pantalla de victoria.

Nota técnica:

```
void pantalla_victoria(long puntaje_final);
```

Tarea 5: main.c — bucle principal

- Llama a pantalla_inicio() al arrancar.
- Bucle for de 4 niveles: cargar_mapa() → inicializar Jugador → bucle de juego → mostrar_resumen_nivel().
- El bucle de juego: leer tecla con getch() → mover_jugador() → imprimir_ventana() + mostrar_hud().
- Al terminar los 4 niveles: llamar calcular_puntaje() de NASM → mostrar_resumen_final() → pantalla_victoria().

Nota técnica:

```
Conecta todo el proyecto en un flujo limpio
```

Punto de integración

- ⇒ Entrega mapas.h con el prototipo de cargar_mapa() el día 1 — desbloquea al Integrante 1.
- ⇒ Los mapas pueden tener diseño simple al inicio y refinarse el día 3.
- ⇒ main.c usa funciones de juego.c (Int. 1) y rutinas.h (Int. 2) — depende de ambos.
- ⇒ Si cargar_mapa() falla, todo el juego falla: probarlo primero con un mapa de 5×5.

Calendario — 4 días

Días 1 y 2: desarrollo paralelo intensivo. Día 3: integración y prueba del flujo completo. Día 4: pulido, commits finales, capturas y entrega del PDF.

Día	Integrante 1	Integrante 2	Integrante 3
Día 1 Desarrollo paralelo	• imprimir_ventana() con mapa hardcodeado • mover_jugador() básico (sin NASM aún) • mostrar_hud()	• rutinas.h con 5 prototipos ← ENTREGAR HOY • contar_caracter() en NASM • validar_movimiento() en NASM	• mapas.h ← ENTREGAR HOY • cargar_mapa() funcional • Mapa nivel1.txt y nivel2.txt (60×60)
Día 2 Desarrollo paralelo	• Lógica de llave, puerta y salida en mover_jugador() • mostrar_resumen_nivel() • mostrar_resumen_final()	• detectar_objeto() en NASM • calcular_puntaje() en NASM • contar_libres() en NASM • build.bat + README.md	• Mapa nivel3.txt y nivel4.txt (60×60) • pantalla_inicio() con arte ASCII • pantalla_victoria()
Día 3 ■ In tegración	• Conectar mapas reales (cargar_mapa) • Conectar las 5 funciones NASM reales • Probar flujo completo nivel 1→4	• Probar build.bat con el proyecto completo • Ajustar convención win64 si hay bugs • Verificar retornos correctos de NASM	• main.c completo: bucle de 4 niveles • Conectar pantalla_inicio / victoria • Probar carga de los 4 mapas .txt
Día 4 Entrega	• Pulir colores y alineación visual • Corregir bugs de movimiento/ventana • Commits finales del motor	• Verificar los 5 retornos con valores reales • Último commit y push a GitHub • Verificar historial de commits	• Capturas de pantalla de los 4 niveles • Reporte PDF completo • Entrega en plataforma

Regla de integración — No bloquearse

Integrante 2 entrega rutinas.h:

```
long contar_caracter(char* mapa, long total, char c);
long validar_movimiento(char* mapa, long cols, long fila, long col);
long calcular_puntaje(long monedas, long pasos, long niveles);
long detectar_objeto(char* mapa, long cols, long fila, long col, char obj);
long contar_libres(char* mapa, long total);
```

Integrante 3 entrega mapas.h:

```
int cargar_mapa(const char* ruta, char mapa[][COLS]);
```

Integrante 1 puede compilar con stubs temporales en juego.c:

```
// stub temporal – reemplazar cuando NASM esté listo
long validar_movimiento(char* m, long c, long f, long col) { return 1; }
```

Historial de commits sugerido

El repositorio debe mostrar avance progresivo. Aquí van los commits mínimos esperados:

#	Mensaje de commit
---	-------------------

1	Estructura inicial: archivos base, headers compartidos, carpeta mapas/
2	Int1: imprimir_ventana() y mover_jugador() básico con mapa hardcodeado
3	Int2: rutinas.asm – contar_caracter() y validar_movimiento()
4	Int3: cargar_mapa() funcional + mapas nivel1.txt y nivel2.txt
5	Int1: lógica de llave, puerta, salida + resúmenes de nivel y final
6	Int2: rutinas.asm – detectar_objeto(), calcular_puntaje(), contar_libres() + build.bat
7	Int3: mapas nivel3.txt y nivel4.txt + pantalla_inicio() y pantalla_victoria()
8	Integración: conectar mapas reales + NASM + main.c completo, prueba flujo nivel 1→4
9	Correcciones de bugs, pulido visual y ajustes de colores
10	Entrega final: README completo, capturas verificadas, historial limpio

Puntos extra contemplados

Mejora	Pts	Responsable	Estado
Colores ANSI en consola	+5	Integrante 1 (colores.h ya definido)	✓ Arquitectura lista
Pantalla de inicio y de victoria	+5	Integrante 3 (main.c)	✓ Contemplado en main.c
Lectura de mapas desde archivos .txt	+5	Integrante 3 (cargar_mapa)	✓ Tarea 2 del Int. 3
Más de 3 niveles funcionales (4 niveles)	+5	Integrante 3 (nivel4.txt)	✓ NUM_NIVELES=4 desde el inicio
Bloques ASCII para mejorar la vista del mapa	+5	Integrante 1 (assets.h ya definido)	✓ assets.h listo
TOTAL puntos extra	+25		